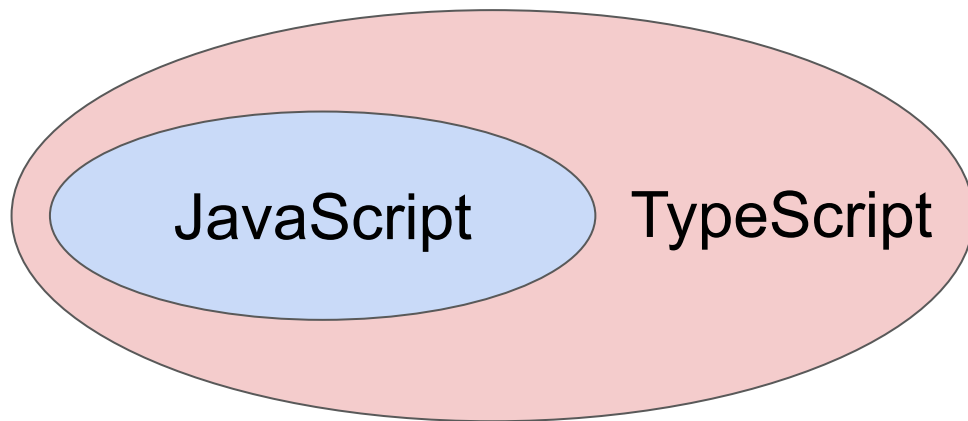


Laboratorio I

a.a. 2025/2026

TypeScript I

TypeScript



<https://www.typescriptlang.org/docs/handbook/>

JavaScript

JavaScript è

- **linguaggio tipizzato**, ma che
- **non prevede controlli statici** sui tipi di dato ed
- **effettua una conversione implicita tra tipi** (*dynamic type-checking*)

JavaScript

JavaScript è

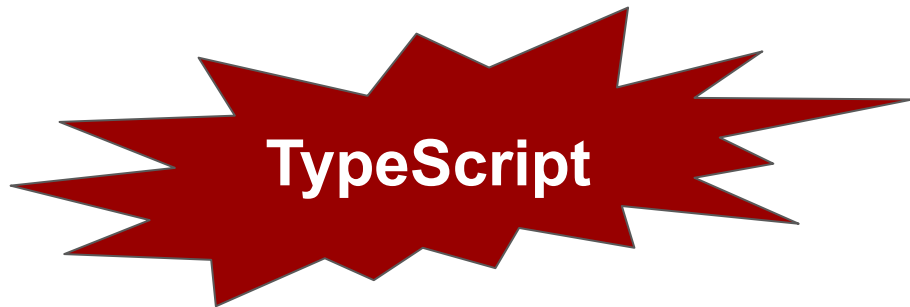
- **linguaggio tipizzato**, ma che
- **non prevede controlli statici** sui tipi di dato ed
- **effettua una conversione implicita tra tipi** (*dynamic type-checking*)



- Per questo JS risulta molto **flessibile** (probabilmente una delle chiavi del suo successo)
- L'assenza di controlli sui tipi durante la *compilazione* può generare **errori difficili** da analizzare a runtime

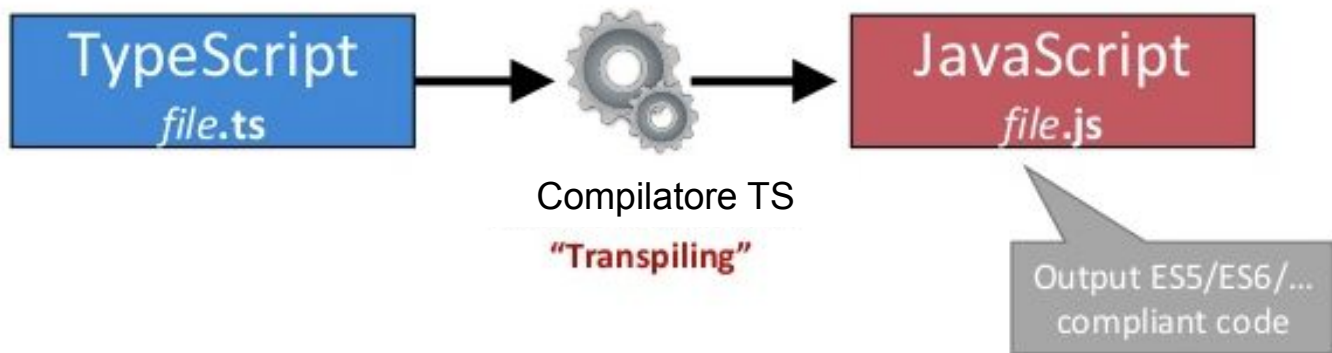
TypeScript

- Per questo, Microsoft ha definito nel 2012 una **estensione** di JavaScript
- Supporto per il controllo statico dei tipi e altre funzionalità pensate per scrittura di applicazioni complesse



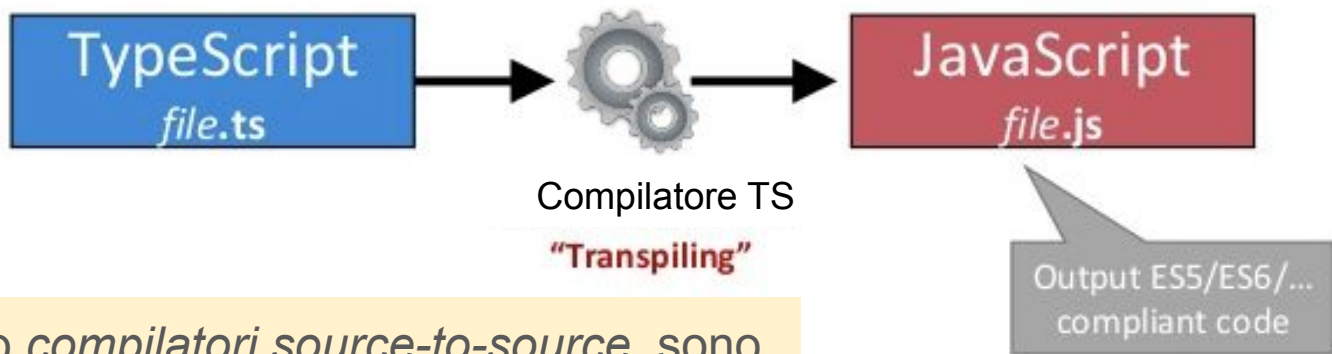
Da JS a TS

- Un'applicazione TypeScript viene tradotta da un "compilatore" (**transpiler**) in un'applicazione JavaScript standard eseguibile su qualsiasi engine



Da JS a TS

- Un'applicazione TypeScript viene tradotta da un "compilatore" (**transpiler**) in un'applicazione JavaScript standard eseguibile su qualsiasi engine



Transpilers, o *compilatori source-to-source*, sono strumenti che leggono codice sorgente in un linguaggio di programmazione e producono codice equivalente in un altro linguaggio che ha un simile livello di astrazione

JS vs TS

- TypeScript Playground

TS

```
1  function somma (a: number, b: number): number {  
2    |   return a+b;  
3  }  
4  
5
```

JS

```
"use strict";  
function somma(a, b) {  
    return a + b;  
}
```


JS vs TS

- TypeScript Playground

TS

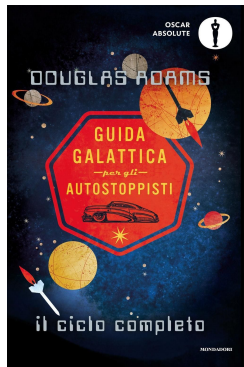
```
6
7 class Greeter {
8   greeting: string;
9
10  constructor(message: string) {
11    this.greeting = message;
12  }
13
14  greet(): string {
15    return "Hello, " + this.greeting;
16  }
17 }
18
19 let greeter: Greeter = new Greeter("world");
```

JS

```
class Greeter {
  constructor(message) {
    this.greeting = message;
  }
  greet() {
    return "Hello, " + this.greeting;
  }
}
let greeter = new Greeter("world");
```

Tipi base

- **boolean**
 - `let flag: boolean = false;`
- **number**
 - `let ultimateQuestionOfLifeUniverseEverything: number = 42;`
- **bigint**
 - `let big: bigint = 100n;`
- **string**
 - `let signoreDegliAnelli3: string = "Il ritorno del re";`
- **array** (dati di tipo **omogeneo**)
 - `let list: number[] = [1, 2, 3];`



Qualcosa di nuovo - tuple

- Tipi Tupla
 - I tipi **tupla** permettono di esprimere
 1. Array con dati **eterogenei**
 2. I tipi da gestire sono indicati al momento della dichiarazione

Qualcosa di nuovo - tuple

- Tipi Tupla
 - I tipi **tupla** permettono di esprimere array eterogenei
 - Ad esempio, tupla con **string** e **number**

```
// Dichiarare un tipo tupla
```

```
let x: [string, number];
```

```
// Inizializza OK
```

```
x = ["hello", 10];
```

```
// Inizializza KO
```

```
x = [10, "hello"]; // Errore
```

```
Type 'number' is not assignable to type 'string'.
```

```
Type 'string' is not assignable to type 'number'.
```

Qualcosa di nuovo - tuple

- Ad esempio, tupla con **string** e **number**

```
// Dichiarare un tipo tupla
```

```
let x: [string, number];
```

```
// Inizializza OK
```

```
x = ["hello", 10];
```

- Accedendo gli elementi tramite indice, viene effettuato il controllo sui tipi

```
// OK
```

```
console.log(x[0].substring(1));
```

```
console.log(x[1].substring(1)); // Errore
```

```
Property 'substring' does not exist on type 'number'.
```

Qualcosa di nuovo - tuple

- Ad esempio, tupla con **string** e **number**

```
// Dichiarare un tipo tupla
let x: [string, number];
// Inizializza OK
x = ["hello", 10];
```

- **Errore** accedendo gli elementi “al di fuori” degli indici “noti”

```
x[3] = "world";    Tuple type '[string, number]' of length '2' has no element at index '3'.
```

```
console.log(x[5].toString());    Object is possibly 'undefined'.
```

```
Tuple type '[string, number]' of length '2'
has no element at index '5'.
```

Qualcosa di nuovo - tuple

- Ad esempio, tupla con **string** e **number**

```
// Dichiarare un tipo tupla
let x: [string, number];
// Inizializza OK
x = ["hello", 10];
```

- Aggiungere elementi?

```
x.push(3); // OK
x.push(true) // KO
```

Argument of type 'boolean' is not assignable to parameter of type 'string | number'.

Qualcosa di nuovo - tuple

- Ad esempio, tupla con **string** e **number**

```
// Dichiarare un tipo tupla
let x: [string, number];
// Inizializza OK
x = ["hello", 10];
```

- Aggiungere elementi?

```
x.push(3); // OK
x.push(true) // KO
```

Essendo tupla un array, è possibile usare le funzioni sugli array che già conosciamo

Argument of type 'boolean' is not assignable to parameter of type 'string | number'.

Qualcosa di nuovo - enum

- Il tipo enum serve per dare nomi a insiemi di valori numerici

```
enum Color {  
    Red, // 0  
    Green, // 1  
    Blue, // 2  
}  
let c: Color = Color.Green; // c vale 1
```

Qualcosa di nuovo - **enum**

- Di default, enum inizia a numerare i suoi elementi da 0. Si può modificare specificando esplicitamente la numerazione

```
enum Color {  
    Red = 1, // 1  
    Green, // 2  
    Blue, // 3  
}  
let c: Color = Color.Green; // c vale 2
```

Qualcosa di nuovo - **enum**

- Di default, enum inizia a numerare i suoi elementi da 0. Si può modificare specificando esplicitamente la numerazione

```
enum Color {  
    Red = 1, // 1  
    Green, // 2  
    Blue, // 3  
}  
let c: Color = Color.Green; // c vale 2
```

Dal nome ottengo la numerazione, ma
è possibile anche il viceversa....

Qualcosa di nuovo - enum

- Ottenere il nome dalla numerazione

```
enum Color {  
    Red = 1,  
    Green,  
    Blue,  
}  
let colorName: string = Color[2];  
console.log(colorName); // mostra 'Green'
```

Dalla numerazione ottengo il numero
Vediamo come è fatto Color:

Qualcosa di nuovo - enum

- Ottenere il nome dalla numerazione

```
enum Color {  
    Red = 1,  
    Green,  
    Blue,  
}  
let colorName: string = Color[2];  
console.log(colorName); // mostra 'Green'
```

Dalla numerazione ottengo il numero
Vediamo come è fatto Color:

```
> Color  
{ '1': 'Red', '2': 'Green', '3': 'Blue', Red: 1, Green: 2, Blue: 3 }  
> 
```

Il tipo `any`

- Quando si dichiara una variabile in JavaScript, stiamo implicitamente affermando che **la variabile potrà contenere qualsiasi tipo di dato**
- Questa assunzione implicita in JavaScript può essere resa esplicitamente in TypeScript specificando il tipo `any`

```
var myVar: any = 123;  
myVar = "Una stringa";  
myVar = true;  
var myArray: any[] = ["stringa", 123, true];
```

`myVar` e `myArray` possono contenere dinamicamente **qualsiasi valore** proprio come avviene nel puro JS

Il tipo `any`

- Quando si dichiara una variabile in JavaScript, stiamo implicitamente affermando che **la variabile potrà contenere qualsiasi tipo di dato**
- Questa assunzione implicita in JavaScript può essere resa esplicitamente in TypeScript specificando il tipo `any`
 - Nessun controllo statico sulle variabili di tipo `any`

```
var myVar: any = 123;  
myVar = "Una stringa";  
myVar = true;  
var myArray: any[] = ["stringa", 123, true];  
var aVar: any = 678;  
aVar.getCodiceFiscale();
```

Nessun controllo statico
sull'esistenza del metodo

Il tipo **any**

- Utile quando si parte da codice JS, per iniziare a inserire il controllo dei tipi nella trasformazione in TS (o viceversa)
- Usando **any** si perde controllo sui tipi
 1. Motivazione principale per usare TS
 2. Quindi meglio evitare se non necessario

Il tipo unknown

- Si riferisce a variabili di cui non conosciamo il tipo staticamente (al tempo della scrittura)
- Esempio simile a `any`

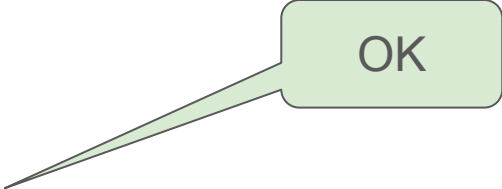
```
let nonSicuro: unknown = 4;  
nonSicuro = "forse una stringa";
```

```
// OK, un boolean  
nonSicuro = false;
```

Il tipo unknown

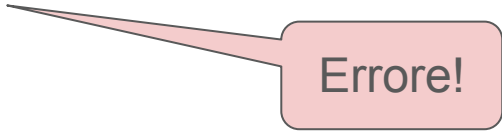
- Si riferisce a variabili di cui non conosciamo il tipo staticamente (al tempo della scrittura)
- A differenza di **any**, **non permette di accedere proprietà che non esistono**
Queste proprietà includono anche funzioni e TS non controlla la loro esistenza o tipo

```
var aVar: any = 678;  
aVar.getCodiceFiscale();
```



OK

```
var bVar: unknown = 678;  
bVar.getCodiceFiscale();
```



Errore!

Il tipo unknown

Non possibile usare variabili di tipo `unknown` in espressioni “tipate” (a meno di inserire espliciti controlli sui tipi)

```
let maybe: unknown;
// 'maybe' potrebbe essere string, object, boolean, undefined, o altro
const aNumber: number = maybe; // KO

if (typeof maybe === "boolean") {
  // TS sa che maybe è boolean
  const aBoolean: boolean = maybe;
  // Quindi, non può essere string
  const aString: string = maybe; // KO
}

if (typeof maybe === "number") {
  // TS sa che maybe è number
  const aNumber: number = maybe;
}

if (typeof maybe === "string") {
  // TS sa che maybe è string
  const aString: string = maybe;
  // Quindi, non può essere boolean
  const aBoolean: boolean = maybe; // KO
}
```

Type 'unknown' is not assignable to
type 'number'

Type 'string' is not assignable to
type 'boolean'

Type 'boolean' is not assignable to
type 'string'

Il tipo `void`

- In un certo senso è l'opposto di `any`
- Indica la mancanza di un tipo, o meglio l'assenza di un valore a cui poter assegnare un tipo
- Si usa per indicare che una funzione non restituisce nulla
 - Ha poco senso usarlo su una variabile, perché potrebbe avere come valore solo `undefined` oppure `null` (se non usato `--strictNullChecks`)

```
function warnUser(): void {  
  console.log("This is my warning message");  
}
```

Attivo di default

```
let inusabile: void = undefined;  
// OK se `--strictNullChecks` non usato  
inusabile = null;
```

Il tipo `null` e `undefined`

- Come facile prevedere, sono i tipi dei valori `null` e `undefined`

```
// Non possiamo fare molto con queste variabili!
```

```
let u: undefined = undefined;
```

```
let n: null = null;
```

- Di default sarebbe possibile assegnare i valori `undefined` e `null` anche a variabili di altro tipo (ad es., `number` o `string`)
 - Non possibile quando `--strictNullChecks` attivo, per evitare appunto errori
- Come fare se vogliamo usare una variabile il cui valore può essere, ad esempio, `string` oppure `null` oppure `undefined`?

Il tipo `null` e `undefined`

- Come facile prevedere, sono i tipi dei valori `null` e `undefined`

`// Non possiamo fare molto con queste variabili!`

`let u: undefined = undefined;`

`let n: null = null;`

- Di default sarebbe possibile assegnare i valori `undefined` e `null` anche a variabili di altro tipo (ad es., `number` o `string`)
 - Non possibile quando `--strictNullChecks` attivo, per evitare appunto errori
- Come fare se vogliamo usare una variabile il cui valore può essere, ad esempio, `string` oppure `null` oppure `undefined`?
 - Potremmo usare `any`.....ma poco TS style!
 - Si usano i **tipi unione**!

I tipi unione

- Il tipo unione descrive un valore che può essere uno fra diversi tipi
- Esempio: `number | string | boolean` denota una variabile che può essere di tipo `number`, `string` oppure `boolean`

```
function padLeft(value: string, padding: string | number) {  
  // ...  
}
```

I tipi unione

```
let prezzo: string | number;
```

```
prezzo = 10.01;  
prezzo = '€2.49';
```

- È anche possibile definire **un nome** per un tipo, *in termini di un altro tipo*, con la keyword **type**

```
// crea un nuovo nome per il tipo 'string | number'  
type Price = string | number;
```

```
let prezzo: Price;
```

```
prezzo = 10.01;  
prezzo = '€2.49';
```


Inferenza di tipo

- TS applica una strategia di **type inference** quando non viene utilizzata alcuna annotazione esplicita sul tipo utilizzato

```
let x = 3;  
// ^ = let x: number
```

Il tipo di x viene “dedotto” dalla inizializzazione

Inferenza di tipo

- TS applica una strategia di **type inference** quando non viene utilizzata alcuna annotazione esplicita sul tipo utilizzato

```
let x = 3;  
// ^ = let x: number
```

```
let x = [0, 1, null];  
// ^ = let x: (number | null)[]
```

L'algoritmo di inferenza dei tipi considera tutti i possibili "tipi candidati" (in questo caso **number** e **null**), e sceglie quello **migliore** (in questo caso il *tipo unione*)

Inferenza di tipo

- TS applica una strategia di **type inference** quando non viene utilizzata alcuna annotazione esplicita sul tipo utilizzato

```
let x = 3;  
// ^ = let x: number
```

```
let x = [0, 1, null];  
// ^ = let x: (number | null)[]
```

- L'inferenza, se necessaria, avviene ad esempio quando si *inizializzano variabili* o quando si determinano *i valori di ritorno di una funzione*

Inferenza di tipo

- TS applica una strategia di **type inference** quando non viene utilizzata alcuna annotazione esplicita sul tipo utilizzato

```
let x = 3;  
// ^ = let x: number
```

```
let x = [0, 1, null];  
// ^ = let x: (number | null)[]
```

Attenzione: per i nostri scopi, dovrete sempre specificare **TUTTI** i tipi!

- L'inferenza, se necessaria, avviene ad esempio quando si *inizializzano variabili* o quando si determinano *i valori di ritorno di una funzione*
- Molto spesso l'inferenza è “ovvia”; in altri casi lo è meno, e TS utilizza svariate regole

Q & A